

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3 – KONSEP DASAR



Balqis Awalunhna Rahmah

103132430011

S1SD-05-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pemrograman Berorientasi Objek (Object-Oriented Programming/OOP) adalah pemrograman berbasis konsep class dan object. Class bertindak sebagai blueprint yang memuat data (field/attribut) dan perilaku (method), sedangkan object adalah instansiasi nyata dari class tersebut. Instance field merupakan variabel milik masing-masing object sehingga nilainya independen. Sebaliknya, class field (dideklarasikan dengan keyword static) adalah variabel milik class yang nilainya digunakan bersama oleh semua object instansiasi class tersebut.

Enkapsulasi menyembunyikan detail internal object dari akses luar menggunakan pembatas akses seperti private untuk field dan public untuk method penjelajah/pengubah. Pada bahasa Java, variabel tipe data primitif (int, double, dsb.) dilewatkan secara pass by value (menyalin nilai asli). Sementara itu, variabel bertipe objek/array dilewatkan secara pass by reference, di mana variabel menyimpan referensi (alamat memori) sehingga perubahan di dalam method akan memengaruhi data asli.

B. Guided

1. Abstraksi

- Abstraksi.java

```
public class Abstraksi {  
    public static void main(String[] args) {  
        Kucing persia = new Kucing();  
  
        persia.nama = "Jeje";  
        persia.ras = "Persia";  
        persia.bersuara();  
    }  
}
```

- Kucing.java

```
public class Kucing {  
    String nama;  
    String ras;  
  
    void bersuara() {  
        System.out.println(nama + ras + "meow");  
    }  
}
```

Output :

```
PS D:\TEL-U PURWOKERTO\SMT 5\PBO\praktikum-pbo-ps1ds-05-reg-01\modul-03-konsep-das  
● ar\project\src\main\java> java Abstraksi  
JejePersiameow
```

2. Class & Object

- Circle.java

```
public class Circle {  
    public static final double PI = 3.14159;  
  
    public static double radiansToDegrees(double rads) {  
        return rads * 180 / PI;  
    }  
  
    public double r;  
  
    public double area() {  
        return PI * r * r;  
    }  
  
    public double circumference() {  
        return 2 * PI * r;  
    }  
}
```

- Main2.java

```
public class Main2 {  
    public static void main(String[] args) {  
        Circle lingkaran = new Circle();  
  
        lingkaran.r = 7;  
  
        System.out.println("Jari-jari : " + lingkaran.r);  
        System.out.println("Luas      : " + lingkaran.area());  
        System.out.println("Keliling  : " + lingkaran.circumference());  
    }  
}
```

Output :

```
PS D:\TEL-U PURWOKERTO\SMT 5\PBO\praktikum-pbo-ps1ds-05-reg-01\modul-03-konsep-das  
● ar\project\src\main\java> java Main2  
Jari-jari : 7.0  
Luas      : 153.93791  
Keliling  : 43.98226
```

3. Enkapsulasi

- Rekening.java

```
public class Rekening {  
    private int saldo = 0;  
  
    public void tambahSaldo(int jumlah) {  
        saldo = saldo + jumlah;  
        System.out.println("Saldo berhasil ditambahkan");  
    }  
}
```

```

    }

    public void tampilkanSaldo() {
        System.out.println("Saldo Anda: " + saldo);
    }
}

```

- Enkapsulasi.java

```

public class Enkapsulasi {
    public static void main(String[] args) {
        Rekening rek = new Rekening();

        rek.tambahSaldo(20000000);
        rek.tampilkanSaldo();
    }
}

```

Output :

```

PS D:\TEL-U PURWOKERTO\SMT 5\PBO\praktikum-pbo-ps1ds-05-reg-01\modul-03-konsep-das
● ar\project\src\main\java> java Enkapsulasi
Saldo berhasil ditambahkan
Saldo Anda: 20000000

```

4. Pass by Value & Pass by Reference

- TestPass.java

```

public class TestPass {
    int i, j;

    TestPass(int a, int b) {
        i = a;
        j = b;
    }

    void calculate(int m, int n) {
        m = m * 10;
        n = n / 2;
    }

    void calculate(TestPass e) {
        e.i = e.i * 10;
        e.j = e.j / 2;
    }
}

```

- PassedByValue.java

```

public class PassedByValue {
    public static void main(String[] args) {
        int x, y;
        TestPass z;
    }
}

```

```

        z = new TestPass(50, 100);

        x = 10;
        y = 20;

        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        z.calculate(x, y);

        System.out.println("Nilai sesudah passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        System.out.println("Nilai sebelum passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);

        z.calculate(z);

        System.out.println("Nilai sesudah passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
    }
}

```

Output :

```

PS D:\TEL-U PURWOKERTO\SMT 5\PBO\praktikum-pbo-psids-05-reg-01\modul-03-konsep-dasar\project\src\main\java> java PassedByValue
Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50

```

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun bukan sesuatu yang berbeda-beda tiap kali data dibuat sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Jawaban :

- `PengolahSuhu.java`

```
public class PengolahSuhu {
    private double[] suhuHarian;
    public static final double NILAI_KOSONG = -1.0;

    public PengolahSuhu(double[] suhuHarian) {
        this.suhuHarian = suhuHarian;
    }

    public void tampilkanData() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.printf("Hari %d : %.1f°C\n", (i + 1), suhuHarian[i]);
            }
        }
    }

    public int cariIndexKosong() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                return i;
            }
        }
        return -1;
    }

    public void isiDataKosong() {
        int index = cariIndexKosong();
        if (index != -1 && index > 0 && index < suhuHarian.length - 1) {
            suhuHarian[index] = (suhuHarian[index - 1] + suhuHarian[index + 1]) /
2.0;
        }
    }
}
```

```

        public double hitungRataRata() {
            double total = 0;
            for (double suhu : suhuHarian) {
                total += suhu;
            }
            return total / suhuHarian.length;
        }
    }

```

- MainUnGuided.java

```

import java.util.Arrays;

public class MainUnGuided {
    public static void main(String[] args) {
        double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();
        System.out.println();

        int indexKosong = pengolah.cariIndexKosong();
        System.out.println("Index hari kosong (dimulai dari 0): " + indexKosong);
        System.out.println();

        pengolah.isiDataKosong();

        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
        System.out.println();

        System.out.printf("Rata-rata : %.2f°C\n\n", pengolah.hitungRataRata());

        System.out.println("Isi array suhuHarian di main setelah isiDataKosong() dijalankan:");
        System.out.println(Arrays.toString(suhuHarian));
    }
}

```

Output :

```
PS D:\TEL-U PURWOKERTO\SMT 5\PBO\praktikum-pbo-pslds-05-reg-01\modul-03-konsep-dasar\project\src\main\java> java MainUngu
ided
Data Suhu Awal
Hari 1 : 30,4°C
Hari 2 : 24,3°C
Hari 3 : 26,8°C
Hari 4 : (kosong)
Hari 5 : 31,4°C
Hari 6 : 30,8°C
Hari 7 : 32,9°C

Index hari kosong (dimulai dari 0): 3

Data Suhu Setelah Pengisian
Hari 1 : 30,4°C
Hari 2 : 24,3°C
Hari 3 : 26,8°C
Hari 4 : 29,1°C
Hari 5 : 31,4°C
Hari 6 : 30,8°C
Hari 7 : 32,9°C

Rata-rata : 29,39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
```

Penjelasan :

Class PengolahSuhu menerapkan enkapsulasi dengan menjadikan suhuHarian bertipe private dan menyediakan class field NILAI_KOSONG (public static final double) sebagai konstanta bernilai -1.0.

Method isiDataKosong() melakukan teknik imputasi data time series dengan mengambil rata-rata suhu satu hari sebelum dan sesudah data yang hilang.

Array suhuHarian pada method main ikut berubah nilainya meskipun tidak diubah langsung di main. Hal ini terjadi karena array di Java bertipe data reference. Saat array dipassing ke constructor PengolahSuhu, constructor menyimpan referensi alamat memori yang sama persis. Perubahan nilai pada elemen array di dalam class PengolahSuhu secara otomatis mengubah isi array pada variabel main.

D. Kesimpulan

Pemahaman mendasar mengenai pengimplementasian Pemrograman Berorientasi Objek menggunakan bahasa Java terutama dalam memisahkan peran antara *class* sebagai cetak biru dan *object* sebagai bentuk instansiasinya dibahas dan dipahami pada modul 3 ini. Menerapkan enkapsulasi dengan pembatas akses private dan public terbukti efektif untuk menjaga keamanan data agar tidak diubah secara sembarangan dari luar kelas.

Selain itu, penggunaan *class field* bertipe static final sangat efisien untuk mendefinisikan konstanta bersama tanpa harus mereplikasi memori pada setiap objek. Pemahaman mengenai sifat tipe data *reference* pada objek dan array juga menjadi poin krusial, sebab manipulasi data yang dilakukan oleh method di dalam kelas akan berdampak langsung pada variabel asli yang dirujuk di program utama.